

equity_derivatives_pricer

Pricer d'options sur actions & desk de market making · Black-Scholes, grecques, volatilité implicite, skew · dashboard Streamlit + terminal CLI

CE QUE FAIT LE PROJET

- Récupère** les données d'options d'une action (SPY, AAPL...) : prix spot, échéances, tous les calls et puts cotés — via Yahoo Finance, ou en mode simulé.
- Nettoie** calcule le prix mid (bid+ask)/2, écarte les options illiquides, convertit l'échéance en maturité en années.
- Valorise** prix théorique Black-Scholes + les 5 grecques (delta, gamma, vega, theta, rho).
- Inverse le modèle** retrouve la **volatilité implicite** de chaque option à partir de son prix de marché (algorithme de Brent).
- Affiche le skew** la courbe volatilité implicite = $f(\text{strike})$, qui montre que le marché ne croit pas à une volatilité constante.
- Tient un desk** cote une fourchette bid/ask, décale son prix selon son stock (inventory skewing), exécute des ordres, se couvre en delta, suit cash et positions.
- Se pilote à 2 endroits** un **dashboard web** (app.py) et un **terminal** (main.py) — même moteur, deux façons de l'utiliser.

LE PIPELINE EN UN COUP D'ŒIL



Chaque couche ignore les autres : le moteur de pricing ne dépend que de NumPy/SciPy (aucun réseau, aucun pandas), et le market maker reçoit un prix déjà calculé. **Preuve que le découpage tient** : app.py et main.py font exactement la même chose métier, sans une seule formule financière dans l'interface.

LES DOSSIERS

Dossier / fichier	Rôle	À quoi ça sert
app.py	INTERFACE WEB	Le dashboard Streamlit : saisie d'ordres, KPI, inventaire, graphe du skew.
main.py	INTERFACE CLI	Le même desk en terminal : boucle de saisie, état affiché après chaque trade.
market_data/	COUCHE DONNÉES	Aller chercher les données de marché et les rendre exploitables.
pricing_engine/	COUCHE MODÈLE	Les mathématiques financières pures. Le cœur du projet.
market_making/	COUCHE TRADING	La logique métier d'un desk : coter, trader, se couvrir.
visualisation/	COUCHE AFFICHAGE	Le tracé du skew en matplotlib, isolé des points d'entrée.
tests/	TESTS	Validation du pricer + diagnostic de connexion Yahoo.

LES CONCEPTS FINANCIERS, EN UNE LIGNE CHACUN

Concept	Traduction en français courant
Option européenne	Le droit d'acheter (call) ou de vendre (put) une action à un prix fixé, à une date fixée.
Black-Scholes	La formule qui donne le prix juste d'une option, en supposant qu'on peut la couvrir en continu.
Delta	De combien le prix de l'option bouge si l'action monte de 1 \$. C'est la grecque de la couverture.
Gamma	À quelle vitesse le delta lui-même change. Gamma élevé = il faut se recouvrir souvent.
Vega	La sensibilité à la volatilité. C'est ce qu'un desk d'options achète et vend vraiment.
Theta	L'érosion du temps : ce que l'option perd juste en vieillissant.
Volatilité implicite	La volatilité qu'il faut injecter dans Black-Scholes pour retomber sur le prix coté.
Skew	L'IV n'est pas la même à tous les strikes : elle baisse quand le strike monte. Le modèle est donc une simplification.
Delta hedging	Vendre des actions pour annuler l'exposition au sens du marché, et ne garder que la volatilité.
Inventory skewing	Baisser son prix quand on a trop de stock, pour inciter le marché à nous en débarrasser.
Exposition brute	La somme des positions en valeur absolue : long 100 / short 100 n'est pas « plat » en risque.
Parité call-put	$C - P = S - K \cdot e^{-rT}$: une égalité qui doit tenir sinon il y a arbitrage.

LES FICHIERS DE CODE, UN PAR UN

Dossier	Fichier	L.	Ce qu'il contient
racine	app.py	158	Le dashboard Streamlit. Formulaire d'ordre en sidebar, 4 KPI (cash, contrats ouverts, couverture delta, spot), table d'inventaire et graphe du skew en thème sombre. Garde l'état du desk entre les clics via <code>st.session_state</code> .
pricing_engine/	black_scholes.py	79	Le cœur. Classe <code>EuropeanOption</code> : prix call/put, les 5 grecques, et <code>implied_volatility()</code> qui inverse le modèle par l'algorithme de Brent. Ne dépend que de <code>numpy</code> + <code>scipy</code> .
market_making/	pricer_mm.py	96	Classe <code>MarketMaker</code> : <code>quote_price()</code> (bid/ask décalé selon le stock), <code>execute_trade()</code> (cash + inventaire), <code>hedge_delta()</code> (couverture en actions). Multi-instruments via dictionnaires.
racine	main.py	51	Le terminal. Boucle interactive : strike, type, sens, ticker, quantité → pricing à l'IV de marché, cotation, exécution, hedge, état du desk. Même logique que <code>app.py</code> .
market_data/	data_fetcher.py	48	Classe <code>MarketDataFetcher</code> : la seule qui touche au réseau. Récupère le spot, les échéances et la chaîne d'options sur Yahoo Finance (<code>yfinance</code>).
market_data/	data_cleaner.py	47	Classe <code>MarketDataCleaner</code> : mid-price, filtre de liquidité (volume > 0 et bid > 0), maturité T en années (jours / 365.25).
market_data/	data_feed.py	23	Jeu de données simulé réaliste (13 strikes autour de 540 \$). Permet de faire tourner la démo sans dépendre de l'API.
market_data/	data_enricher.py	21	Ajoute la colonne IV à toute la chaîne d'options. C'est le pont entre la couche données et le moteur de pricing.
tests/	test_black_scholes.py	22	Valide le pricer par la parité call-put ($C - P = S - K \cdot e^{-rT}$), une relation d'arbitrage qui ne dépend d'aucun modèle. Passe.
tests/	test_fetcher.py	15	Script de diagnostic : vérifie que la connexion Yahoo Finance répond. Pas un test <code>pytest</code> au sens strict.
visualisation/	plots.py	9	<code>plot_volatility_skew()</code> : trace l'IV en fonction du strike (calls bleu, puts rouge). Utilisé par <code>main.py</code> .

CE QUI SE PASSE QUAND ON PASSE UN ORDRE

- Lecture du carnet** on retrouve la ligne du strike demandé dans la chaîne nettoyée (sa maturité T et sa vol implicite).
- Pricing** l'option est repricée avec **sa propre IV de marché**, pas une vol arbitraire — on cote autour du marché.
- Cotation** `quote_price()` renvoie bid/ask autour du prix théorique, décalés selon le stock déjà détenu sur cet instrument.
- Exécution** `execute_trade()` met à jour cash et inventaire, indexés par un identifiant du type `AAPL_call_540`.
- Couverture** $\text{delta agrégé} = \text{inventaire} \times \text{delta unitaire} \rightarrow \text{hedge_delta}()$ achète ou vend les actions manquantes.

OÙ EN EST LE PROJET	STACK TECHNIQUE
OK Pipeline complet et fonctionnel dans les deux interfaces. Le skew calculé est réaliste (30 % à K=480 → 16 % à K=590). Le test <code>pytest</code> passe. Plus aucune duplication de code. BUG Le sens Sell est inversé (<code>app.py</code> + <code>main.py</code>) : <code>-quantity</code> et <code>action_client</code> s'annulent, donc le desk ne peut pas se mettre short et vendre lui coûte du cash. Correctif : une ligne. À FAIRE Ajouter <code>streamlit</code> à <code>requirements.txt</code> , un <code>.lower()</code> sur la saisie du type dans <code>main.py</code> , et un README avec capture du dashboard.	streamlit Dashboard web interactif numpy / scipy Calcul et optimisation (brentq) pandas Manipulation des chaînes d'options matplotlib Tracé du skew yfinance Données de marché pytest Tests automatisés

Lancement du dashboard : `streamlit run app.py` | Terminal : `python main.py` | Tests : `pytest tests/`